

Python Fundamentals

Variables, Arithmetic Operators & Reflection Exercises

IDE: Visual Studio Code • Language: Python 3.x • Level: Beginner

Overview

This worksheet guides you through hands-on Python exercises using **Visual Studio Code (VS Code)** as your integrated development environment. You will practice declaring and updating variables, applying all seven arithmetic operators, and reflecting on the behavior you observe — building the mental models that underpin all future Python programming.



Open VS Code → open a New Terminal (Ctrl + ` on Windows/Linux, Cmd + ` on Mac) → type `python3` and press Enter to launch the interactive Python shell. All exercises are completed in this shell unless stated otherwise.

Part 1 — Key Concepts

1.1 What Is a Variable?

A variable is a named storage location in memory. In Python you create (declare) a variable simply by assigning a value to a name:

```
age = 16           # integer
name = "Alex"      # string
temperature = 98.6 # float (decimal)
```

Key rules: variable names must start with a letter or underscore; they are case-sensitive (`score` ≠ `Score`); use descriptive names in snake_case convention.

1.2 Arithmetic Operators — Quick Reference

Python supports seven arithmetic operators. Study this table before starting the exercises:

Operator	Symbol	Example	Result	Notes
Addition	+	5 + 3	8	Combines two values
Subtraction	-	10 - 4	6	Finds the difference
Multiplication	*	6 * 7	42	Repeated addition
Division	/	15 / 4	3.75	Returns a float
Integer Div.	//	15 // 4	3	Drops the decimal (floor)
Modulo	%	15 % 4	3	Returns the remainder

Exponent	**	3 ** 3	27	Raises to a power
----------	----	--------	----	-------------------



Python follows standard order of operations (PEMDAS/BODMAS). Use parentheses to override: `result = (2 + 3) * 4` gives 20, while `2 + 3 * 4` gives 14.

Part 2 — Guided Exercises

Work through each exercise in the VS Code Python shell. After completing the steps, answer the reflection questions in writing or in discussion with your class.

Exercise 1 — Declaring Variables & Basic Addition

Learning Objectives

- Declare integer variables
- Use the + operator
- Observe how `print()` displays results

Expected Output:

```
20
<class 'int'>
```

Steps (in VS Code Terminal)

1. In the terminal, type: `number1 = 12` and press Enter.
2. Type: `number2 = 8` and press Enter.
3. Type: `sum_result = number1 + number2` and press Enter.
4. Type: `print(sum_result)` and press Enter.
5. Now try: `print(type(sum_result))` — note the data type shown.

Reflection

- *What value was displayed? Is it what you expected?*
- *What does `type()` tell you about `sum_result`? Why does that matter?*
- *What happens if you change `number1 = 12.0`? Does the output change?*

Exercise 2 — Subtraction & Updating Variables

Learning Objectives

- Use the - operator
- Reassign a variable in place
- Use += and -= shorthand

Expected Output:

```
14
20
30
58
55
```

Steps (in VS Code Terminal)

6. Declare: `initial_value = 25` and `reduction = 11`.
7. Calculate: `difference = initial_value - reduction` then print it.
8. Now update `initial_value` using: `initial_value -= 5` then print `initial_value`.
9. Use `initial_value += 10` and print again.
10. Create a counter = 50, increment by 8 using +=, then decrement by 3 using -=. Print after each step.

 Reflection

- What is the difference between `initial_value = initial_value - 5` and `initial_value -= 5`?
- In what real-life programs might you need to increment or decrement a variable repeatedly?
- What happens if you try: `25 - 'five'`? What error appears, and why?

Exercise 3 — Multiplication & Order of Operations**Learning Objectives**

- Use the `*` operator
- Understand operator precedence
- Test parentheses impact

Expected Output:

```
13
25
63
18.75
```

Steps (in VS Code Terminal)

11. Declare: `x = 5, y = 2, z = 3`.
12. Type: `result_a = x * y + z` then print `result_a`.
13. Type: `result_b = x * (y + z)` then print `result_b`.
14. Declare: `multiplier = 7` and `multiplicand = 9`. Calculate and print their product.
15. Calculate: `item_price = 3.75` and `quantity = 5`. Compute total cost and print it.

 Reflection

- Why are `result_a` and `result_b` different even though they use the same values?
- In a real shopping app, why is it important that multiplication happens before addition?
- What would happen if Python evaluated operators strictly left-to-right with no precedence rules?

Exercise 4 — Division, Integer Division & Modulo**Learning Objectives**

- Distinguish `/` from `//`
- Use `%` to find remainders
- Connect modulo to real-world logic

Expected Output:

```
5.0
5
4.285...
4
3
0
1
1
40
```

Steps (in VS Code Terminal)

16. Declare: `numerator = 30` and `denominator = 6`. Compute and print both `numerator / denominator` and `numerator // denominator`.
17. Change denominator to 7. Re-run both division expressions. Notice the difference.
18. Declare: `value = 19` and `modulus = 4`. Compute: `remainder_val = value % modulus`. Print it.
19. Try: `print(20 % 2)` and `print(21 % 2)`. What do these tell you about even/odd numbers?
20. Compute: `print(100 // 60)` and `print(100 % 60)`. Think: if 100 minutes elapsed, how many full hours and leftover minutes is that?

 Reflection

- When would you choose `//` over `/`? Give a concrete programming example.
- How could you use `%` to check if a number is even or odd in a program?
- What result does `0 % 5` give? What about `5 % 5`? Can you form a rule?

Exercise 5 — Exponents & Geometric Calculations

Learning Objectives

- Use the `**` operator
- Combine variables in formulas
- Replicate real mathematical formulae

Expected Output:

```
27
120
46
153.938...
78.53975
41.46...
```

Steps (in VS Code Terminal)

21. Declare: `base_num = 3` and `power_num = 3`. Compute: `exponent_result = base_num ** power_num`. Print it.
22. Declare: `length = 15` and `width = 8`. Calculate and print `rectangle_area = length * width`.
23. Calculate and print: `rectangle_perimeter = 2 * (length + width)`.
24. Declare: `radius = 7` and `pi = 3.14159`. Calculate: `circle_area = pi * radius ** 2`. Print it.
25. Change `radius = 5`. Recalculate `circle_area` without redefining `pi`. Print the new result.
26. Calculate: `area_difference = rectangle_area - circle_area`. Print it.

Reflection

- Why do we store `pi` as a variable instead of typing `3.14159` every time?
- What does `radius ** 2` compute? Why is the exponent applied before the multiplication by `pi`?
- How could you modify the circle formula to compute circumference instead of area?

Exercise 6 — Combining All Operators — Challenge

Learning Objectives

- Chain multiple operations
- Apply all 7 arithmetic operators
- Predict output before running

Expected Output:

```
13
15
7
5.0
13
```

Steps (in VS Code Terminal)

27. Before typing anything, predict the result of each expression below. Write your prediction on paper.
28. Type: `print(2 + 3 * 4 - 1)` — check your prediction.
29. Type: `print((2 + 3) * (4 - 1))` — check your prediction.
30. Type: `print(17 // 3 + 17 % 3)` — check your prediction.
31. Type: `print(2 ** 4 / 4 + 1)` — check your prediction.

32. Create a variable: `base_amount = 10`. Apply:
`base_amount = base_amount * 2 - 7`. Print
`final_amount`.

Reflection

- How many of your predictions were correct before running the code? What caused any errors in your thinking?
- Describe in plain English what `17 // 3 + 17 % 3` is actually calculating.
- Write a single Python expression (no loops or conditionals) that computes the area of a ring (annulus): the area between a circle of radius 7 and a circle of radius 3.

Part 3 — Extended Reflection

Answer the following questions in full sentences. Use examples from your own exercise work to support each answer.

Q1	In your own words, explain what a variable is and why programmers use them instead of typing literal numbers everywhere in their code.
Q2	Describe a situation — in a game, app, or daily life — where each of the seven arithmetic operators would be useful. Give a concrete example for each.
Q3	What is operator precedence, and why does Python (and mathematics) need it? What would go wrong without it?
Q4	Explain the difference between <code>/</code> (true division) and <code>//</code> (integer division). When would a programmer choose one over the other?
Q5	You used <code>+=</code> and <code>-=</code> in Exercise 2. How do these shorthand operators improve code readability and reduce mistakes?
Q6	Reflect on the experience of predicting output before running code (Exercise 6). How did this strategy help you understand operator precedence more deeply?

Part 4 — Extension Challenges

Completed all exercises early? Try these open-ended challenges. Use the VS Code file editor (not the shell) and save your work as a `.py` file.

Challenge A

Temperature Converter: Declare `fahrenheit = 98.6`. Write a single expression to convert it to Celsius using the formula $C = (F - 32) \times 5 / 9$ and print the result with a descriptive label using an f-string: `print(f'{fahrenheit}°F = {celsius:.2f}°C')`.

**Challenge
B**

Pizza Party: A pizza has 8 slices. You have 5 friends (6 people total). Write code that calculates how many full slices each person gets (use `//`) and how many slices remain (use `%`). Print both answers.

**Challenge
C**

Compound Interest: Declare `principal = 1000`, `rate = 0.05` (5%), and `years = 3`. Calculate compound interest using: `amount = principal * (1 + rate) ** years`. Print the final amount rounded to 2 decimal places.

VS Code Quick Reference

Essential VS Code Actions for This Worksheet		
Open Terminal	<code>Ctrl + `</code> (Win/Linux) <code>Cmd + `</code> (Mac)	Opens the integrated terminal panel at the bottom
Launch Python Shell	Type: <code>python3</code> then Enter	Starts an interactive Python REPL session
New Python File	<code>Ctrl+N</code> → save as <code>file.py</code>	Use for Extension Challenges
Run Python File	<code>Ctrl+F5</code> or right-click → Run Python	Executes the entire <code>.py</code> file
Exit Python Shell	Type: <code>exit()</code> then Enter	Returns you to the standard terminal prompt
Clear Terminal	<code>Ctrl+K</code> (Win/Linux/Mac)	Clears terminal output — doesn't affect variables